ContextOS Project Kya Hai Aur Kaise Kaam Karta Hai? (Asaan Shabdon Mein)

Imagine karo ek badi company hai (jaise Google, Swiggy, ya koi Bank). Unke paas hazaron internal documents hain:

- Engineering designs & architecture
- Incident postmortems (bug fixes)
- Slack chats & Jira tickets
- **Aur confidential cheezein bhi:** Employees ki salary, bank statements, CEO ke private notes.

Ab company chahti hai ki unke engineers aur managers **ek AI (ChatGPT jaise) se sawaal pooch sakein** aur unko instant sahi jawaab mile.

🚨 Problem Kya Aati Hai? (Bina ContextOS Ke)

1. **Security / Data Leak Risk:** Agar koi junior intern AI se pooch le: *"Tell me the CEO's salary and bonus?"* — agar direct AI laga hoga toh AI private salary document padhkar leak kar dega!
2. **Hallucination & Token Limits:** Agar aap direct poori file AI ko bhejoge, toh token limit khatam ho jayegi aur AI ulte-seedhe man-ghadant (hallucinated) jawaab dene lagega.

🛡️ ContextOS Kya Karta Hai? (The Universal Context Layer)

ContextOS ek Security Guard aur Brain ki tarah beech mein baithta hai. AI ke paas koi bhi document jaane se pehle ContextOS 5 stages se use filter karta hai:

```
[User ka Sawaal]
      ⬇
1. Hybrid Search (pgvector + Keyword Search)
      ⬇
2. Retrieval-Time ACL Filter (Security Guard: Kaun pooch raha hai?)
      ⬇
3. Deterministic Reranker (Sabse important kaun sa hai?)
      ⬇
4. Context Assembler (XML format + Token Budget limit)
      ⬇
[Safe & Accurate Prompt AI ko jata hai]
```

🔍 Har Ek Stage Ka Detail:

1. Ingestion Pipeline & Embeddings (Data store kaise hota hai?)

- Company ke documents (Markdown, GitHub code, PDFs) ko hum chhote-chhote **Chunks** (pieces) mein todte hain.
- Har chunk ko **1536-dimensional Vector** (mathematical coordinates) mein convert karke **PostgreSQL ke** `pgvector` extension mein store karte hain.

2. Hybrid Search (pgvector + Full-Text Search)

Kewal vector search kafi nahi hota, kyunki agar query mein exact error code ya ID ho (jaise `APOLLO-421`), toh vector search confuse ho sakta hai.

- **pgvector:** Meaning (sense) samajhta hai (Cosine Distance `<=>`).
- **PostgreSQL Full-Text Search (`plainto_tsquery`):** Exact words aur IDs match karta hai.
- Dono ke results ko merge karke top candidates nikaale jaate hain.

3. Retrieval-Time ACL (Access Control List) — *Superpower Feature*

Yeh is project ka sabse bada USP hai!

- Jab search hoti hai, ContextOS check karta hai: *"Sawaal poochne wale ka role kya hai?"*
- Agar role `ROLE_ENGINEER` hai, toh Engineering docs dikhenge, lekin Executive Salary doc **chupke se filter** ho jayega.
- Agar role `ROLE_EXECUTIVE` hai, toh unko salary data bhi dikhega.
- **AI ko wahi data milta hai jo us user ko dekhne ki permission hai! Zero data leak!**

4. Deterministic Reranker & XML Context Assembly

- Bachi hui verified documents ko rank kiya jata hai (Score: Vector + Keyword + Freshness).
- Phir ek strict **Token Budget** (e.g., 2000 tokens) ke andar unhe XML tag mein wrap kiya jata hai:

```xml
<retrieved_document source="GITHUB" id="APOLLO-421" title="...">
   Incident root cause was webhook timeout...
</retrieved_document>
```

- AI ko context milta hai with exact citations (source link ke saath), jisse hallucination impossible ho jati hai.

5. Redis Caching & Telemetry (Next.js Dashboard)

- Agar wahi sawaal dobara poocha jaye, toh database query kiye bina `< 5ms` mein Redis cache se serve hota hai.
- **Frontend Dashboard (`http://localhost:3000/retrieval-tracer`):** Real-time mein dikhata hai ki kaun se documents select hue, kaun se ACL ne block kiye, aur kitna latency laga!

🎯 Interview ya Presentation mein 30 Seconds mein kya bolna hai?

> *"ContextOS ek Multi-Tenant Enterprise Context Engine hai jo RAG (Retrieval-Augmented Generation) applications ke liye secure context provide karta hai. Isme humne PostgreSQL pgvector aur Full-Text search ka hybrid retrieval banaya hai, jisme retrieval-time Access Control List (ACL) permission filtering hoti hai taaki AI kabhi company ka sensitive data leak na kare. Yeh poora system Java 21, Spring Boot 3, Redis, aur Next.js 14 par bana hai."*